

Operating Systems and System Programming

Practical – 5

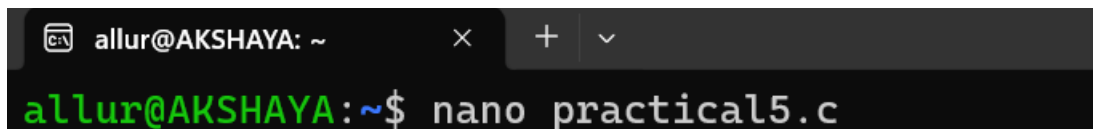
Aim

To implement producer-consumer communication using an anonymous pipe, where the parent process generates data and the child process consumes it, measure communication time, and execute the command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Algorithm

1. Start the program.
2. Create an anonymous pipe using `pipe()`.
3. Create a child process using `fork()`.
4. The parent writes data into the pipe, and the child reads and displays it.
5. Measure and display the communication time.
6. Create another pipe for executing `ls -l | grep ".c"`.
7. Create two child processes using `fork()`.
8. Redirect the output of `ls` to the pipe using `dup2()`.
9. Redirect the input of `grep` from the pipe using `dup2()`.
10. Execute `ls -l` and `grep ".c"` using `exec()`.
11. The parent waits for both child processes using `waitpid()`.
12. Close the pipes and terminate the program.

Step-1:

A terminal window with a dark background. The title bar shows 'allur@AKSHAYA: ~' and window control buttons. The prompt is 'allur@AKSHAYA:~\$' and the command 'nano practical5.c' is being entered.

Step-2:Code

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical5.c *  
  
#include <stdio.h>  
#include <unistd.h>  
#include <string.h>  
#include <sys/wait.h>  
#include <time.h>  
  
int main()  
{  
    int fd[2];  
    pid_t pid;  
    char message[] = "Hello from Parent";  
    char buffer[100];  
    struct timespec start, end;  
  
    // Task 1: Producer-Consumer  
    pipe(fd);  
    pid = fork();  
  
    if (pid == 0)  
    {  
        close(fd[1]);  
        read(fd[0], buffer, sizeof(buffer));  
        printf("Child received: %s\n", buffer);  
        close(fd[0]);  
        return 0;  
    }  
    else  
    {  
        close(fd[0]);  
        clock_gettime(CLOCK_MONOTONIC, &start);  
        write(fd[1], message, strlen(message) + 1);  
        close(fd[1]);  
        wait(NULL);  
        clock_gettime(CLOCK_MONOTONIC, &end);  
  
        double time = (end.tv_sec - start.tv_sec) * 1e6  
            + (end.tv_nsec - start.tv_nsec) / 1e3;  
  
        printf("Parent sent: %s\n", message);  
        printf("Communication time: %.2f microseconds\n", time);  
    }  
  
    // Task 2: ls -l | grep ".c"  
    printf("\nFiles containing .c:\n");  
  
    pipe(fd);  
  
    pid_t p1 = fork();  
  
    if (p1 == 0)  
    {  
        close(fd[0]);  
        dup2(fd[1], STDOUT_FILENO);  
        close(fd[1]);  
    }  
}
```

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical5.c *  
  
    }  
    else  
    {  
        close(fd[0]);  
        clock_gettime(CLOCK_MONOTONIC, &start);  
        write(fd[1], message, strlen(message) + 1);  
        close(fd[1]);  
        wait(NULL);  
        clock_gettime(CLOCK_MONOTONIC, &end);  
  
        double time = (end.tv_sec - start.tv_sec) * 1e6  
            + (end.tv_nsec - start.tv_nsec) / 1e3;  
  
        printf("Parent sent: %s\n", message);  
        printf("Communication time: %.2f microseconds\n", time);  
    }  
  
    // Task 2: ls -l | grep ".c"  
    printf("\nFiles containing .c:\n");  
  
    pipe(fd);  
  
    pid_t p1 = fork();  
  
    if (p1 == 0)  
    {  
        close(fd[0]);  
        dup2(fd[1], STDOUT_FILENO);  
        close(fd[1]);  
    }  
}
```

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 practical5.c *  
    close(fd[1]);  
  
    execlp("ls", "ls", "-l", NULL);  
    perror("ls");  
    return 1;  
}  
  
pid_t p2 = fork();  
  
if (p2 == 0)  
{  
    close(fd[1]);  
    dup2(fd[0], STDIN_FILENO);  
    close(fd[0]);  
  
    execlp("grep", "grep", ".c", NULL);  
    perror("grep");  
    return 1;  
}  
  
close(fd[0]);  
close(fd[1]);  
  
waitpid(p1, NULL, 0);  
waitpid(p2, NULL, 0);  
  
return 0;  
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/ Go To Line	M-

Step-3: Compile and Run

```

allur@AKSHAYA:~$ nano practical5.c
allur@AKSHAYA:~$ gcc practical5.c -o practical5
allur@AKSHAYA:~$ ./practical5
Child received: Hello from Parent
Parent sent: Hello from Parent
Communication time: 451.52 microseconds

Files containing .c:
drwxr-xr-x 2 allur allur 4096 Sep 22 14:35 alarm-clock
-rwxr-xr-x 1 allur allur 36416 Sep 23 18:44 alarm_clock
-rw-r--r-- 1 allur allur 35471 Sep 23 18:44 alarm_clock.c
-rw-r--r-- 1 allur allur 35268 Sep 23 18:57 alarm_clock.c.save
prw-r--r-- 1 allur allur 0 Sep 23 18:44 alarm_control_fifo
drwxr-xr-x 2 allur allur 4096 Sep 24 10:26 alarmclock
-rw-r--r-- 1 allur allur 1004 Jul 30 11:01 command
-rw----- 1 allur allur 1004 Jul 30 10:57 command.save
-rw----- 1 allur allur 1004 Jul 30 10:58 command.save.1
-rw----- 1 allur allur 1004 Jul 30 10:59 command.save.2
-rw-r--r-- 1 allur allur 1004 Jul 30 11:02 command_exec.c
-rw----- 1 allur allur 1004 Jul 30 11:00 command_exec.c.save
-rw-r--r-- 1 allur allur 173 Aug 23 12:09 exec.c
-rw-r--r-- 1 allur allur 157 Aug 11 19:58 fork.c
-rw-r--r-- 1 allur allur 818 Aug 18 11:38 fork_demo.c
-rw-r--r-- 1 allur allur 348 Sep 19 15:07 prac-4.c
-rw-r--r-- 1 allur allur 623 Aug 22 13:07 practical-2.c
-rw-r--r-- 1 allur allur 336 Sep 19 14:49 practical-4
-rw-r--r-- 1 allur allur 531 Sep 19 14:39 practical-4.c
-rw-r--r-- 1 allur allur 815 Aug 22 20:23 practical3.c
-rw-r--r-- 1 allur allur 336 Sep 19 14:53 practical4_task2.c
-rwxr-xr-x 1 allur allur 16560 Oct 1 08:49 practical5
-rw-r--r-- 1 allur allur 1582 Oct 1 08:49 practical5.c
-rwxr-xr-x 1 allur allur 16472 Jul 30 11:04 process
-rw-r--r-- 1 allur allur 1004 Jul 30 11:04 process.c
-rw----- 1 allur allur 1004 Jul 30 10:56 process.c.save
-rw-r--r-- 1 allur allur 75 Aug 11 20:35 read.c
-rw-r--r-- 1 allur allur 266 Sep 21 13:45 simplecommand.c
-rw-r--r-- 1 allur allur 0 Aug 22 21:19 skill1.1.c
-rw-r--r-- 1 allur allur 373 Aug 23 12:09 skill1.c
-rw-r--r-- 1 allur allur 1218 Aug 23 13:08 skill2-2.c
-rw-r--r-- 1 allur allur 299 Aug 23 12:34 skill2.c
-rw-r--r-- 1 allur allur 1165 Aug 24 13:48 skill3-2.c
-rw-r--r-- 1 allur allur 2082 Aug 24 12:39 skill3.c
-rwxr-xr-x 1 allur allur 16176 Oct 1 08:29 skill4
-rwxr-xr-x 1 allur allur 16176 Oct 1 08:33 skill4-2
-rw-r--r-- 1 allur allur 681 Oct 1 08:33 skill4-2.c
-rw-r--r-- 1 allur allur 525 Oct 1 08:29 skill4.c
-rw-r--r-- 1 allur allur 43 Aug 22 13:03 source.txt
-rw-r--r-- 1 allur allur 1004 Aug 1 15:04 task-1.c
-rw-r--r-- 1 allur allur 348 Sep 19 14:49 task-2.c
allur@AKSHAYA:~$

```

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe, and the communication time was measured. The command `ls -l | grep ".c"` was successfully executed using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.